



ONDOKUZ MAYIS ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ

GEOINTEL İLE MCP SERVER GELİŞTİRİLMESİ

PROJE RAPORU

Eyüphan BİNİCİ

Murat GÜMÜŞ

Danışman

Dr. Öğr. Üyesi Oğuz Emre KURAL

Aralık, 2025

TEŞEKKÜR

Bu proje kapsamında gerçekleştirdiğimiz çalışmalar sürecinde tecrübelerini bizimle paylaşan ve akademik rehberliğiyle çalışmanın şekillenmesinde en büyük katkıyı sağlayan değerli danışman hocamız Dr. Öğr. Üyesi Oğuz Emre KURAL'a en içten teşekkürlerimizi sunarız.

Ayrıca bu projenin literatür altyapısını oluştururken faydalandığımız, daha önce bu alanda emek vermiş ve bilgi birikimlerini paylaşarak bilim dünyasına katkı sağlamış tüm araştırmacılara ve bilim insanlarına teşekkür ederiz.

Ö Z E T

Bu projede, mevcut navigasyon uygulamalarının sunduğu statik yönlendirme hizmetindeki kısıtları aşarak, kullanıcılara bağlama duyarlı ve kişiselleştirilmiş bir deneyim sunan yeni nesil bir Coğrafi Bilgi Sistemi (GIS) mimarisinin tasarımı ve geliştirilmesi planlanmıştır.

Proje, kullanıcının isteklerini analiz ederek hava durumu, trafik yoğunluğu, yerel şehir verileri ve çevresel faktörleri birleştiren ve buna göre optimum kararı otonom olarak verebilen yapay zeka tabanlı bir yapı sunmayı hedeflemektedir.

Proje kapsamında, Büyük Dil Modellerinin (LLM) mekansal veriler üzerindeki muhakeme yeteneğini arttırmak ve halüsinasyon riskini minimize etmek amacıyla Model Context Protocol (MCP) standardı benimsenecektir. Uydu görüntülerinin işlenmesinde ise Cloud-Native yaklaşımı tercih edilmiş, STAC ve COG standartları sayesinde büyük boyutlu uydu verilerini indirmek yerine gerekli analizlerin anlık olarak yapılması hedeflenmiştir.

Sistemin mekansal veritabanı yönetimi PostgreSQL üzerine çalışan PostGIS eklentisi ile sağlanacaktır. Otonom araçların çok adımlı planlama ve hata yönetimi süreçleri için LangGraph kütüphanesi kullanılacaktır .

Mobil uygulamanın geliştirilmesinde, yüksek performanslı harita görselleştirmesi sağlamak amacıyla Flutter framework'ü ve Impeller rendering motoru kullanılacaktır. Backend mimarisi, asenkron yapıyı destekleyen Python tabanlı FastAPI üzerinde kurgulanacaktır.

Bu proje, akıllı şehir verilerini otonom ajan teknolojileriyle birleştirerek, kentsel hareketliliği sadece hız odaklı değil, insan ve konfor odaklı bir deneyime dönüştürmeyi hedeflemektedir.

İÇİNDEKİLER

KISALTMALAR DİZİNİ	viii
I GİRİŞ	
1 AMAÇ	2
2 LİTERATÜR ÖZETİ	3
2.1 Büyük Dil Modelleri (LLM) ve Mekansal Zeka	3
2.1.1 LLM’lerin Gelişimi	3
2.1.2 Spatial RAG ve Agentic GIS	4
2.2 Model Context Protocol (MCP)	7
2.2.1 MCP Standartları ve Çalışma Prensipleri	7
2.3 Cloud-Native Mekansal Bilişim	8
2.3.1 STAC (Spatio Temporal Asset Catalog) Spesifikasyonları	9
2.3.2 COG (Cloud Optimized GeoTIFF) ve Streaming Teknolojileri	10
2.4 Mevcut Çözümlerin Analizi	11
II MATERYAL	
3 VERİ KAYNAĞI VE API ALTYAPISI	14
3.1 Global Servis Sağlayıcılar	14
3.1.1 Rotalama Veri Kaynakları	16
3.1.2 Meteorolojik Veri Kaynakları	17
3.1.3 İlgi Noktaları (POI) Kaynakları	18

3.2	Yerel ve Dinamik Veri Kaynakları	19
3.2.1	Yerel Yönetim Verileri	19
3.2.2	Çevresel Uydu Verileri	19
3.2.3	Yapısal Olmayan Veriler	20
4	YAZILIM VE TEKNOLOJİ YIĞINI	21
4.1	Veri Saklama ve Yönetim Katmanı	21
4.1.1	İlişkisel ve Mantıksal Veritabanı: PostgreSQL	21
4.1.2	Mekansal İndeksleme Mekanizması (GIST)	22
4.1.3	Anlamsal Arama ve Vektör Veritabanı: pgvector	22
4.1.4	Dinamik Rota Hesaplama Motoru: pgRouting	23
4.2	Backend Yapısı	24
4.2.1	Python ve FastAPI Asenkron Mimarisi	24
4.2.2	Veri Doğrulama: Pydantic	24
4.2.3	Ajan Orkestrasyonu ve Durum Yönetimi: LangGraph	25
4.2.4	Dinamik Veri Kazıma Teknolojileri: Headless Browsers	26
4.3	İletişim ve Entegrasyon Protokolleri	26
4.3.1	Model Context Protocol (MCP) Uygulaması	26
4.3.2	Cloud-Native Veri Akışı	27
4.4	Mobil İstemci Teknolojileri	27
4.4.1	Flutter Mimari Avantajlar	28
4.4.2	Skia'dan Impeller'a Geçiş	28
4.4.3	Harita Performansı	29
4.5	Altyapı, Dağıtım ve Ölçeklenebilirlik	30
4.5.1	Konteyner Orkestrasyonu(Kubernetes)	30

III YÖNTEM

5	SİSTEM MİMARISI VE UYGULAMA YÖNTEMİ	32
5.1	Genel Sistem Mimarisi	32
5.1.1	Hibrit MCP Mimarisi	33
5.2	Yapay Zeka ve Orkestrasyon	35
5.2.1	Model Seçimi	35
5.2.2	LangGraph ile Dinamik Yönlendirme	35
5.3	Geliştirilen Algoritmalar ve Karar Mekanizmaları	37
5.3.1	Çevresel Faktörlü Rota Optimizasyonu	37
5.3.2	Mekan Filtreleme ve Hibrit Mekansal Arama	39
5.3.3	WFS Veri Dönüşümü	40
5.3.4	Uydu Görüntüsü İşleme (NDVI)	41
5.4	Uygulama Bileşenleri: MCP Sunucuları	41
5.4.1	Çekirdek GIS Sunucusu	41
5.4.2	Uydu Analiz Sunucusu	42
5.4.3	İzole Web Kazıma (Scraping) Sunucusu	42

IV SONUÇ

6	SONUÇ	45
	KAYNAKÇA	47

ŞEKİLLER LİSTESİ

2.1	Geleneksel RAG Mimarisi ile Spatial RAG Mimarisinin Karşılaştırılması	4
2.2	LLM Tabanlı Agentic GIS Mimarilerinde Problem Çözme[1]	6
2.3	Geleneksel İndirme Yöntemi ile COG Streaming Yöntemi Karşılaştırma	10
4.1	Köprü Tabanlı Mimari ile Flutter Mimarisinin Karşılaştırılması	27
4.2	Kubernetes Tabanlı PostGIS Küme Mimarisi	30
5.1	Genel Sistem Mimarisi ve Veri Akış Şeması	32
5.2	LangGraph ile Döngüsel İş Akışı	36
5.3	Web Verilerinin Çekilmesi Süreci	42

ÇİZELGELER LİSTESİ

2.1	Literatürde incelenen mevcut MCP sunucularının karşılaştırmalı analizi.	12
3.1	Kullanılan veri kaynakları, sağlayıcılar ve entegrasyon protokolleri.	15
4.1	Flutter ve React Native mimarilerinin harita render performanslarının karşılaştırması [2].	29
5.1	Projede kullanılan MCP taşıma mekanizmalarının teknik karşılaştırması.	34
6.1	Proje İş Zaman Çizelgesi	46

KISALTMALAR DİZİNİ

AOT	: Ahead-of-Time (Derleme Zamanı)
API	: Application Programming Interface (Uygulama Programlama Arayüzü)
ASGI	: Asynchronous Server Gateway Interface (Asenkron Sunucu Ağ Geçidi)
COG	: Cloud Optimized GeoTIFF (Bulut İçin Optimize Edilmiş GeoTIFF)
DOM	: Document Object Model (Belge Nesne Modeli)
GIS	: Geographic Information Systems (Coğrafi Bilgi Sistemleri)
GiST	: Generalized Search Tree (Genelleştirilmiş Arama Ağacı)
LLM	: Large Language Models (Büyük Dil Modelleri)
MCP	: Model Context Protocol
NDVI	: Normalized Difference Vegetation Index (Normalize Fark Bitki Örtüsü)
NLP	: Natural Language Processing (Doğal Dil İşleme)
OGC	: Open Geospatial Consortium
ORS	: OpenRouteService
OSM	: OpenStreetMap
OSRM	: Open Source Routing Machine
POI	: Point of Interest (İlgi Noktası)
RAG	: Retrieval-Augmented Generation (Erişim Artırılmış Üretim)
SSE	: Server-Sent Events (Sunucu Tarafından Gönderilen Olaylar)
STAC	: Spatio Temporal Asset Catalog (Mekansal Zamansal Varlık Kataloğu)
VGI	: Volunteered Geographic Information (Gönüllü Coğrafi Bilgi)
WFS	: Web Feature Service
WSGI	: Web Server Gateway Interface (Web Sunucusu Ağ Geçidi Arabirimi)

BÖLÜM: İ

GİRİŞ

A M A Ç

Mevcut navigasyon uygulamalarının sunduğu tek boyutlu ve birbirinden kopuk veri sunumunu aşarak, farklı kaynaklardan gelen verileri bütünleşik bir bağlamda yorumlayabilen otonom bir sistem geliştirmek amaçlanmaktadır. Geleneksel navigasyon uygulamaları hava durumu, trafik ve kullanıcı tercihlerini izole katmanlar olarak sunarken; bu proje, Model Context Protocol (MCP) mimarisi sayesinde kullanıcı adına en iyi kararı veren ajan tabanlı bir yapı sunmayı hedeflemektedir.

Büyük Dil Modellerinin (LLM) muhakeme yeteneğini, coğrafi bilgi sistemlerinin analitik gücüyle birleştirmek amaçlanmıştır. Bu entegrasyondaki en kritik araç olan MCP, küresel ölçekteki verilerle yerel ölçekteki verileri ortak bir zeminde buluşturmaktadır. Bu sayede sistem, sadece "En kısa yol neresi?" sorusuna değil, "Şu anki yağış durumu ve trafik yoğunluğuna göre gitmem gereken en güvenli ve mantıklı yol hangisi?" gibi karmaşık ve çok değişkenli sorulara yanıt verebilecek bir yetkinliğe ulaşacaktır.

LİTERATÜR ÖZETİ

2.1 *Büyük Dil Modelleri (LLM) ve Mekansal Zeka*

2.1.1 *LLM'lerin Gelişimi*

LLM'ler, doğal dil işleme (NLP) alanında önemli yetenekler sergilese de, mekansal verilerin işlenmesi konusunda yapısal kısıtlılıklar barındırmaktadır. Geleneksel LLM'ler, eğitim verilerindeki metin örüntülerini taklit ederek çıktı üretirler. Ancak coğrafi veriler metin tabanlı değildir ve matematiksel kesinlik gerektirir. LLM'ler, olasılıksal doğaları gereği, deterministik mekansal hesaplamalarda başarısız olma eğilimindedir. Örneğin, Claude 3, GPT-4 ve Gemini gibi modellerin mekansal görevlerdeki performansını karşılaştıran çalışmalar, bu modellerin kod yorumlama ve temel Coğrafi Bilgi Sistemleri (Geographic Information Systems - GIS) teorisi sorularında başarılı olduğunu, ancak haritalama ve karmaşık mekansal muhakeme gerektiren görevlerde zayıf kaldığını ortaya koymuştur[3].

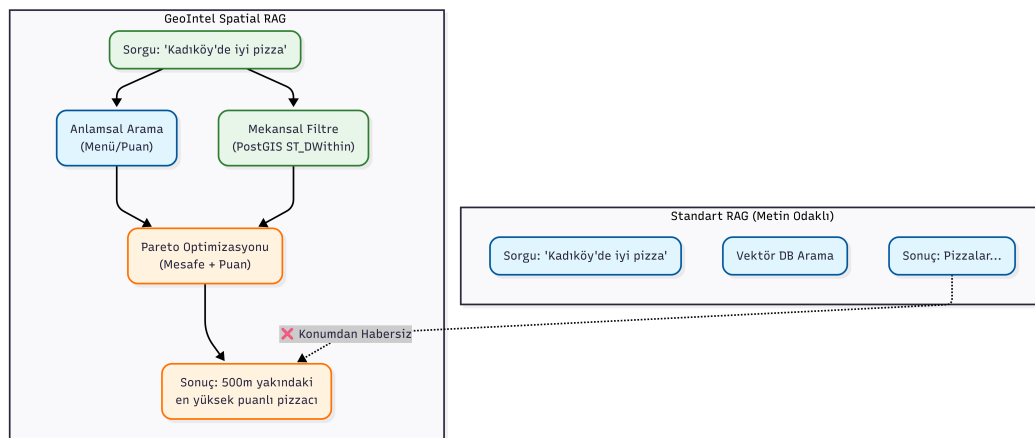
Bir çalışmada, GPT-4o'nun rota planlama görevlerinde başlangıçta sadece %12.4 başarı gösterdiği, ancak Düşünce Zinciri stratejisi uygulandığında bu oranın %87.5'e yükseldiği

görülmüştür [4]. Bu durum, modellerin saf haliyle mekansal zekaya sahip olmadığını, ancak doğru yönlendirme ve araçlarla bu yeteneği kazanabileceğini göstermektedir.

Ajan tabanlı sistemlerin başarısı, temel aldıkları LLM'ler ile doğrudan ilişkilidir. Ancak, LLM'ler halüsinasyon adı verilen, akıcı ve ikna edici fakat olgusal olarak yanlış bilgiler üretme eğilimindedir. Bu durum, özellikle coğrafi analiz, navigasyon ve mekansal sorgulama gibi doğruluğun hayati olduğu alanlarda kritik bir engeldir [5].

Halüsinasyonlar, modelin eğitim verisindeki önyargılardan, mimari kısıtlamalardan veya çıkarım sırasındaki stokastik süreçlerden kaynaklanabilir. Modeller, gerçek doğruluk yerine, dilbilgisel ve anlamsal olabilirlik üzerine optimize edildikleri için, istatistiksel desenleri gerçeğin kendisiymiş gibi sunabilirler. Mekansal bağlamda bu, var olmayan bir binanın tarif edilmesi veya yanlış bir coğrafi ilişkinin kurulması şeklinde ortaya çıkabilir. Bu sorunu azaltmak için, modelin içsel bilgisine güvenmek yerine, harici ve doğrulanmış bilgi kaynaklarına başvurmasını sağlayan RAG (Retrieval-Augmented Generation) mimarileri geliştirilmiştir[6].

2.1.2 Spatial RAG ve Agentic GIS



Şekil 2.1: Geleneksel RAG Mimarisi ile Spatial RAG Mimarisinin Karşılaştırılması

RAG mimarileri, modelin bir soruyu cevaplamaadan önce harici bir bilgi tabanından ilgili dokümanları çekmesini ve bunları bağlam olarak kullanmasını sağlar[7].

Standart RAG sistemleri, metin tabanlı benzerlik araması yaparak bilgi getirir, ancak mekansal ilişkileri anlamakta yetersiz kalır. Örneğin, "Müzeeye yakın iyi bir restoran" sorgusunda standart RAG, restoranın kalitesiyle ilgili metinleri bulabilir ancak yakınlık kavramını geometrik olarak işleyemez. Spatial RAG çerçevesi, bu eksiği gidermek için hibrit bir erişim mekanizması önerir:

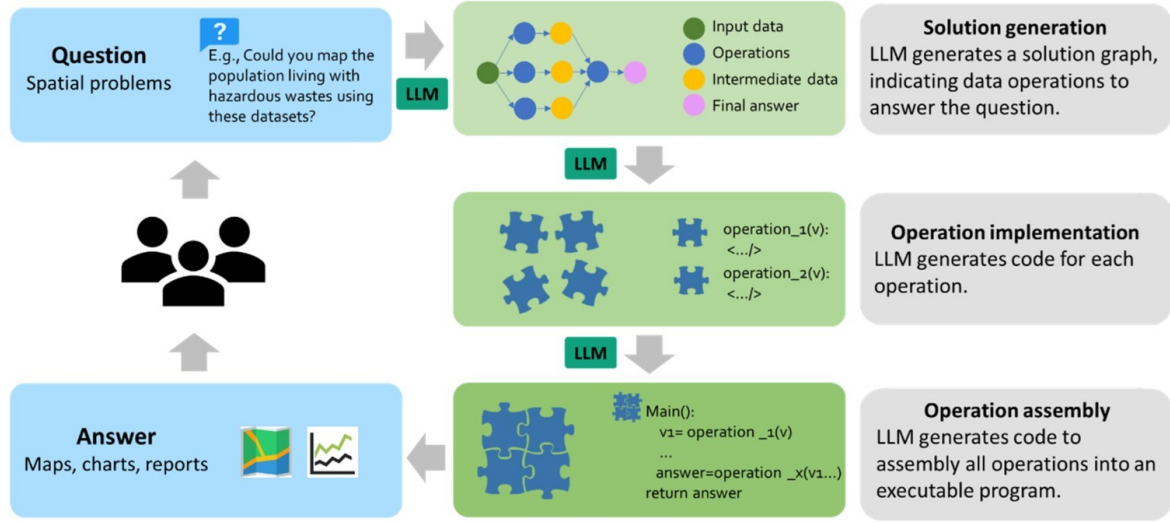
- Seyrek Mekansal Erişim (Sparse Spatial Retrieval): Mekansal veritabanları veya SQL sorguları kullanılarak yapılan, kesin geometrik kısıtlamaları içeren filtrelemelerdir.
- Yoğun Anlamsal Erişim (Dense Semantic Retrieval): LLM tabanlı vektör gömüleri kullanılarak yapılan anlamsal benzerlik aramalarıdır.

Bu iki mekanizma, Çok Amaçlı Optimizasyon stratejisi çerçevesinde entegre edilir. Sistem, aday sonuçları hem mekansal yakınlık hem de anlamsal uygunluk açısından değerlendirerek bir Pareto Sınırı oluşturur. Bu sayede, kullanıcıya hem mekansal kısıtlamaları sağlayan hem de anlamsal isteklere en uygun olan dengeli sonuçlar sunulur. Yapılan deneyler, Spatial-RAG'in standart yöntemlere göre mekansal soru-cevap görevlerinde (GeoQA) doğruluk, kesinlik ve sıralama performansını önemli ölçüde artırdığını göstermektedir [8].

Geleneksel GIS otomasyonu, önceden tanımlanmış kuralların adım adım işletilmesine dayanır. Buna karşılık, son yıllarda literatüre giren "Agentic GIS" kavramı, belirsizlik altında karar verebilen ve plan yapabilen otonom sistemleri ifade eder. Bu ajanlar, algılama-muhakeme-eylem (Perception-Reasoning-Action) döngüsünü otonom olarak sürdürebilir.

Agentic GIS mimarilerinin temelinde, "ReAct" (Reason + Act) olarak adlandırılan bir prompt mühendisliği tekniği yatar. Bu teknik, modelin önce ne yapacağını düşünmesini

örneğin; "Kullanıcı trafik durumunu sordu, önce trafik API'sine gitmeliyim", ardından işlemi gerçekleştirmesini ve sonucu gözlemlemesini sağlar.



Şekil 2.2: LLM Tabanlı Agentic GIS Mimarilerinde Problem Çözme[1]

Sistem bu döngüyü şu adımlarla işletir:

- Soruyu analiz eder ve gerekli veri setlerini belirler.
- İlgili veri kaynaklarına (API'ler, veritabanları) bağlanır ve veriyi çeker.
- Uygun analiz modellerini (regresyon, kümeleme) seçer ve çalıştırır.
- Sonuçları haritalandırır ve yorumlayarak kullanıcıya sunar.

2.2 Model Context Protocol (MCP)

Model Context Protocol (MCP), Anthropic tarafından 2024 yılında tanıtılan ve açık kaynak topluluğu tarafından hızla benimsenen, yapay zeka modellerinin dış dünya ile iletişimini standardize eden bir protokoldür. MCP, yapay zeka uygulamaları için bir "USB-C portu" işlevi görerek, herhangi bir uyumlu uygulamanın herhangi bir uyumlu veri kaynağına bağlanmasını sağlar. Bu standartlaşma, sadece geliştirme sürecini hızlandırmakla kalmaz, aynı zamanda daha güvenilir, ölçeklenebilir ve bağlam farkındalığı yüksek yapay zeka ajanlarının oluşturulmasına olanak tanır [9].

2.2.1 MCP Standartları ve Çalışma Prensipleri

MCP ekosistemi dört ana bileşenden oluşur.

- MCP Host (Ana Bilgisayar): LLM'i çalıştıran ve kullanıcıyla etkileşime giren uygulamadır.
- MCP Client (İstemci): Host ortamı içinde çalışan ve sunucularla bağlantıyı yöneten protokol katmanıdır. MCP sunucusu ile birebir iletişim bağlantısını sürdürür.
- MCP Server (Sunucu): Dış araçları, veri kaynaklarını ve iş akışlarını yöneten, yapay zeka modellerine gerekli kaynakları sağlayan bağımsız servistir.
- Transport Layer (Taşıma Katmanı): İstemci ve sunucu arasındaki veri akışını sağlayan kanaldır [10].

MCP Sunucuları, yeteneklerini üç temel primitive üzerinden sunar:

- Araçlar (Tools): Modelin çağırabileceği yürütülebilir fonksiyonlardır. Bu araçlar, karmaşık işlemleri sunucu tarafında yürüterek modelin halüsinasyon görme riskini azaltır.
- Kaynaklar (Resources): Modelin okuyabileceği pasif veri varlıklarıdır (örneğin veritabanı kayıtları). Kaynaklar URI şemalarıyla adreslenir.
- İstemler (Prompts): Modelin belirli görevleri yerine getirmesine yardımcı olmak için önceden hazırlanmış şablonlardır.

MCP, ajanların sosyal yetenek ve proaktiflik boyutlarını güçlendirir. Ajanların birlikte çalışma düzeni, merkezi bir yönetimden ziyade tamamen dağıtık ve otonom bir yapıya doğru evrilmektedir.. MCP, ajanların ortak bir dil ve referans çerçevesi kullanmasını sağlayarak, merkezi olmayan koordinasyonu kolaylaştırır. Bu sayede, ajanlar görevleri yeteneklerine göre dinamik olarak paylaşabilir, karmaşık problemleri alt görevlere bölebilir ve sonuçları birleştirebilir [11].

2.3 Cloud-Native Mekansal Bilişim

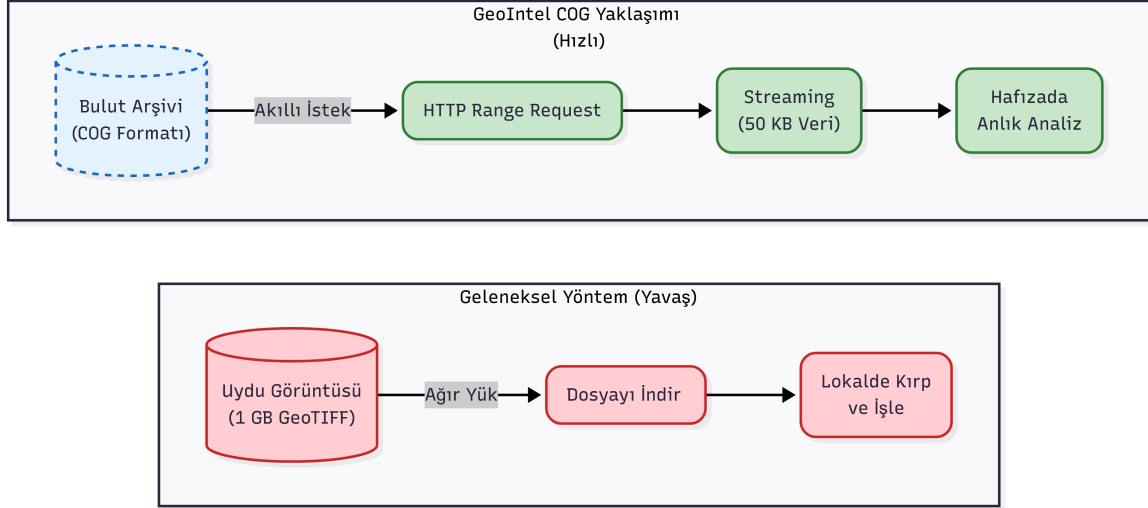
Uydu görüntüleri ve raster verilerin (mekansal veri işleme bilimlerinde kullanılan piksel temelli bir veri modeli) boyutları petabayt seviyelerine ulaştığında, geleneksel indir ve işle yöntemi sürdürülemez hale gelmiştir. Verinin bulunduğu yerde işlenmesini sağlayan "Cloud Native Geospatial" yaklaşımı, büyük veri setlerinin bulut tabanlı platformlarda, veri küplerine benzer yapılarla yönetilmesini ve analiz edilmesini temel alır [12].

2.3.1 STAC (*Spatio Temporal Asset Catalog*) Spesifikasyonları

Coğrafi verileri zaman ve mekan ekseninde indeksleyen, JSON tabanlı modern bir standarttır. STAC, verinin kendisini değil, veriye dair meta veriyi standartlaştırarak farklı sağlayıcılardan gelen verilerin keşfedilebilirliğini artırır. Bir STAC yapısı dört ana bileşenden oluşur [13]:

1. STAC Öğesi: Temel atomik birimdir. Bir GeoJSON özelliği olarak temsil edilir ve belirli bir zaman/mekandaki varlığı tanımlar.
2. STAC Katalog: Öğeleri gruplayan esnek bir JSON yapısıdır.
3. STAC Koleksiyon: Kataloğun genişletilmiş halidir; lisans, sağlayıcı ve uzamsal kapsam gibi ek bilgiler içerir.
4. STAC API: Öğelerin aranmasını sağlayan RESTful uç noktadır. STAC'ın en büyük başarısı, küçük ve esnek bir çekirdek üzerine inşa edilip topluluk tarafından geliştirilmesiyle açıklanmaktadır.

2.3.2 COG (Cloud Optimized GeoTIFF) ve Streaming Teknolojileri



Şekil 2.3: Geleneksel İndirme Yöntemi ile COG Streaming Yöntemi Karşılaştırma

Veriyi bulmak sorunun sadece yarısıdır; diğer yarısı ise veriye verimli erişimdir. Geleneksel GeoTIFF dosyaları, tek bir pikseli okumak için bile dosyanın tamamının indirilmesini gerektirebilir. Cloud Optimized GeoTIFF (COG), dosya iç yapısının bulut tabanlı okuma işlemleri için optimize edildiği yenilikçi bir formattır. OGC (Open Geospatial Consortium) tarafından standartlaştırılan bu yapı, büyük raster dosyalarının akış (streaming) mantığıyla okunmasını sağlar [14]. COG'un çalışma prensibi iki temel teknolojiye dayanır:

1. Dahili Organizasyon: COG dosyalarında, dosyanın meta verisini tutan IFD (Image File Directory) dosyanın en başına yerleştirilir. Görüntü verisi ise küçük karolara bölünür ve farklı çözünürlüklerdeki önizlemeler dosya içinde saklanır.
2. HTTP Range Requests: İstemci, sunucudan dosyanın tamamını istemek yerine, sadece ihtiyacı olan bayt aralığını talep eder.

Bu yapı, Streaming teknolojisini mümkün kılar. Otonom ajan, 1 GB'lık bir uydu görüntüsünün tamamını indirmek yerine, sadece analiz edeceği parkın üzerine denk gelen 50 KB'lık veri parçasını talep eder. Sunucu, sadece bu parçayı gönderir. Bu yerinde işleme yeteneği, bant genişliği kullanımını yüksek oranda azaltır, veri kopyalama ihtiyacını ortadan kaldırır ve analiz süresini dakikalardan milisaniyelere düşürür [14].

2.4 Mevcut Çözümlerin Analizi

Mevcut literatürde en yaygın görülen ilk yaklaşım, teknoloji devlerinin sunduğu tescilli API servislerinin MCP standardına uyarlanarak yapay zeka modellerine sunulmasıdır. Google Maps Grounding Lite [15] ve Mapbox Developer MCP [16] mimari yaklaşıma verilebilecek en tipik örneklerdir. Bu yaklaşım, LLM'lerin en büyük problemi olan halüsinasyon sorununu çözmek adına, verinin doğruluğunun kaynağında garanti altına alındığı Topraklama mekanizması açısından büyük avantaj sağlar [17]. Ajan, var olmayan bir adresi uydurmak yerine bu servislerden teyitli bilgi alır. Ancak bu yaklaşım, literatürde Kapalı Kutu sorunu yaratmaktadır [18].

İkinci yaklaşım, açık kaynaklı verilerin işlenmeden veya sadece görselleştirme amacıyla kullanıldığı mimaridir. Bu kategoride OSM-MCP projesinin Python ve Go dillerindeki farklı uygulamaları öne çıkmaktadır. Bu mimarinin temel kısıtı kalıcılık ve derinliktir. Go tabanlı sistemler veritabanı barındırmadığı için kalıcı depolama sunmaz [19]. Python sürümü ise tarayıcı tabanlı görselleştirme ile sınırlı kalmakta, ajana veriyi işleyip yeni bilgi üretme imkanı tanımamaktadır [20].

Literatürdeki üçüncü yaklaşım, aracı katmanları en aza indirerek veritabanı ve masaüstü yazılım yeteneklerinin filtresiz biçimde ajana sunulmasıdır. Veri katmanında PostGIS MCP

[21] ve CARTO MCP [22], analitik süreçleri doğrudan veritabanı şeması üzerinden yürüterek ajana bir SQL arayüzü sunar. Masaüstü GIS tarafında ise QGIS [23] ve ArcGIS Pro [24] yerel işletim sistemi seviyesinde süreçler arası iletişim kurarak LLM ile entegre olur. Ancak bu mimari, ajanın teknik detaylarla boğuşmasını gerektirir [25]. Bu durum, ajanı fiilen bir veritabanı yöneticisine dönüştürme riskini taşır. Ajanın veritabanı topolojisini ve sütun yapılarını ezberlemek zorunda kalması, modelin bağlam penceresini verimsiz kullanmasına yol açar [26].

Çizelge 2.1: Literatürde incelenen mevcut MCP sunucularının karşılaştırmalı analizi.

Çözüm / MCP	Teknoloji	Temel Yetenek	Kısıt / Yaklaşım
Sunucusu			
Google Maps Grounding	Google API	Halüsinasyonsuz yer verisi ve Knowledge Graph erişimi	Merkeziyetçi yapı ve ticari API bağımlılığı
Mapbox Developer	Node.js	Statik harita üretimi ve Matrix API ile rota hesabı	Analizden ziyade görselleştirme odaklı yapı
OSM-MCP (Python)	Flask (Python)	SSE ile harita üzerine anlık işaretleme yapabilme	Tarayıcı tabanlı görselleştirme ile sınırlı, performans düşük
OSM-MCP (Go)	Go (Golang)	Yüksek eşzamanlılık ve bellek içi hızlı geometrik işlemler	Veritabanı barındırmaz, kalıcı depolama yoktur
PostGIS MCP	PostgreSQL	Veritabanı şemasını okuma ve pgvector ile anlamsal arama	Ajanı "DB Yöneticisi" gibi kullanır, otonomluk kısıtlıdır
QGIS MCP	Python Plugin	TCP üzerinden masaüstü arayüzünü kontrol etme	Arka planda çalışan QGIS uygulamasına mecburdur
ArcGIS Pro MCP	C# / .NET	Named Pipes ile hızlı iletişim ve katman/zoom kontrolü	Sadece Windows'ta çalışır ve lisanslı yazılım gerektirir

BÖLÜM: İİ

MATERYAL

VERİ KAYNAĞI VE API ALTYAPISI

3.1 *Global Servis Sağlayıcılar*

Geliştirilen sistemin veri altyapısı, tek bir kaynağa bağımlı kalmadan, farklı disiplinlerden (trafik, meteoroloji, mekansal veri) beslenen çok katmanlı ve hibrit bir mimari üzerine inşa edilmiştir. Bu bölümde, sistemin karar destek mekanizmalarını besleyen dış servisler ve veri entegrasyon yöntemleri detaylandırılmaktadır. Servis sağlayıcı seçimlerinde; verinin mekansal ve zamansal çözünürlüğü, API yanıt süreleri, küresel kapsam ve entegrasyon maliyetleri belirleyici kriterler olmuştur. Ticari servislerin sunduğu yüksek veri kalitesi ile açık kaynaklı çözümlerin esnekliğini birleştiren bu yapıda kullanılan veri kategorileri, ilgili sağlayıcılar ve iletişim teknolojileri Tablo 3.1’de özetlenmiştir.

Çizelge 3.1: Kullanılan veri kaynakları, sağlayıcılar ve entegrasyon protokolleri.

Veri Kategorisi	Sağlayıcı / Kaynak	Teknoloji	Kullanım Amacı ve Özellik
Rotalama ve Trafik	HERE Maps Routing API v8 ve OSRM	REST API	Araç sensör verilerine dayalı trafik yoğunluğu
Meteorolojik Veri	OpenWeatherMap	One Call API 3.0	Dakikalık yağış tahminleri
İlgi Noktaları (POI)	Google Places & OpenStreetMap (Hibrit)	REST API & PBF	Ticari güncellik ve fiziksel detayların birleştirilmesi
Yerel Veriler	İBB Açık Veri Portalı	WFS	Dinamik şehir verileri
Uydu Görüntüleri	Sentinel-2 (Microsoft Planetary Computer)	STAC & COG	İndirme yapmadan analizler için bulut yerel erişim
Yapısal Olmayan Veri	Web Siteleri	Headless Browser	API desteği olmayan dinamik verilerin anlık çekilmesi

3.1.1 Rotalama Veri Kaynakları

Navigasyon modülünün omurgasını oluşturan rota hesaplama ve trafik yoğunluk verileri için projemizde hibrit bir veri yapısı benimsenmiştir. Bu mimari, küresel ölçekte endüstriyel güvenilirliği kanıtlanmış olan HERE Maps Routing API ile yerel ölçekte kullanacağımız OpenStreetMap (OSM) tabanlı çözümlerin entegrasyonundan oluşur. Bu tercihin temelinde, verinin toplanma metodolojisindeki yapısal farklılıklar ve endüstriyel güvenilirlik yatmaktadır.

Literatürdeki karşılaştırmalı analizler, Google Maps'in trafik verisini büyük oranda son kullanıcıların mobil cihazlarından gelen GPS sinyallerine dayandığını göstermektedir [27]. Buna karşılık HERE Maps, veriyi doğrudan otomotiv endüstrisindeki araçların dahili sensörlerinden beslenen bir yapı üzerine kurmuştur. Akademik çalışmalar, bağlı araçların silecek hızı, ABS aktivasyonu ve tekerlek dönüş hızı verilerinin, sadece konum bildiren GPS verilerine kıyasla çevresel koşulları algılamada daha yüksek hassasiyet sağladığını kanıtlamaktadır [28]. Özellikle Q-Learning tabanlı trafik sinyalizasyon algoritmaları üzerine yapılan güncel çalışmalar, HERE Maps verilerinin anlık trafik sıkışıklığı tespitinde, fiziksel sensör kurulumuna gerek kalmadan yüksek doğruluk sağladığını raporlamaktadır[29]. Proje kapsamında bu küresel servis sağlayıcı, açık kaynaklı rotalama motorlarıyla (ORS/OSRM) entegre edilerek hibrit bir yapıda kullanılacaktır. Bu strateji, sistemin pilot bölgelerde kendi yol grafi üzerinden özgün maliyet fonksiyonları çalıştırmasına olanak tanırken; küresel ölçekteki sorgularda ise OSRM'nin 'Daratılmış Hiyerarşiler' (Contraction Hierarchies) algoritması çok yüksek RAM maliyeti oluşturacağı için [30] HERE platformunun ölçeklenebilir bulut API'si kullanılarak donanım maliyetlerini optimize etmek planlanmaktadır[31].

3.1.2 Meteorolojik Veri Kaynakları

Kentsel hareketlilikte çevresel faktörlerin rota optimizasyonuna dahil edilmesi, projenin temel özgün değerlerinden biridir. Bu kapsamda, meteorolojik veri sağlayıcı olarak OpenWeatherMap One Call API 3.0 seçilmiştir.

Akademik literatürde yapılan deneysel çalışmalar, yağışın trafik akış parametreleri üzerindeki büyük etkisini nicel olarak ortaya koymuştur. Virginia'daki otoyol segmentleri üzerinde yapılan bir araştırma, hafif yağmurun (0.01-0.25 inç/saat) yol kapasitesini %4 ile %10 oranında, şiddetli yağmurun (>0.25 inç/saat) ise %25 ila %30 oranında düşürdüğünü kanıtlamıştır. Ayrıca, yağışın varlığı, şiddetinden bağımsız olarak ortalama işletme hızında %3-5 oranında bir azalmaya yol açmaktadır [32]. İstanbul'da O2, O3, O4 Otoyolları ve D100 Karayolu segmentleri üzerinde yapılan bir araştırmada ise, yağmurun etkisi altında otoyollarda serbest akım hızının %8,61, D100 Karayolu'nda ise % 8,16 oranında azaldığını kanıtlamıştır. Yağışın varlığı yol kapasitesinde otoyollarda %7,88, D100 Karayolu'nda ise %9,07 oranında bir düşüşe yol açmaktadır [33].

Brezilya'daki meteorolojik istasyonlar (INMET) ile yapılan karşılaştırmalı doğruluk analizleri, OpenWeatherMap verilerinin sıcaklık ve nem tahminlerinde ($R^2 > 0.8$) yerel sensör ağlarıyla yüksek korelasyon gösterdiğini doğrulamıştır. [34] Sistemin kullandığı API, Dakikalık Tahmin özelliği sayesinde, ajanların yerel hava olaylarına karşı proaktif rota değiştirmesine olanak tanır.

3.1.3 İlgi Noktaları (POI) Kaynakları

Mekansal analizlerde kullanılan İlgi Çekici Nokta (Point of Interest - POI) verilerinin kaynağı, sistemin güvenilirliği ve kapsayıcılığı açısından kritiktir. Projede, tek bir kaynağın önyargılarını elimine etmek amacıyla OpenStreetMap (OSM) ve Google Places verilerini birleştiren hibrit bir yaklaşım benimsenmiştir.

Avrupa'daki dört farklı kentsel bağlamda (Versilia, Gothenburg, Nice ve Viyana) yapılan "15 Dakikalık Şehir" modellemesi çalışmaları, Google Maps verilerinin ticari yoğunluğun yüksek olduğu merkezlerde çok güçlü olduğunu, ancak merkezden uzaklaştıkça algoritmik filtrelemeler nedeniyle veri setinde yapay bir seyrekleşme yaşandığını göstermektedir. Google API'sinin belirli bir alanda döndürdüğü POI sayısını sınırlaması (yaklaşık 60 adet), mekansal analizlerde eksik veri riskini doğurur [35].

Buna karşılık, Gönüllü Coğrafi Bilgi tabanlı olan OSM, veri yoğunluğu açısından bazı kategorilerde Google'ın gerisinde kalsa da, mekansal örnekleme açısından daha tutarlı ve içsel tutarlılığı yüksek bir yapı sergilemektedir [36].

Bu nedenle sistemde, temel keşif ve kamusal alan sorgularında maliyet etkinliği ve tarafsızlığı nedeniyle OSM verisini; ticari işletmelerin açılış saati, anlık yoğunluk gibi dinamik detay verilerinde ise Google Places verisini kullanılacaktır.

3.2 *Yerel ve Dinamik Veri Kaynakları*

3.2.1 *Yerel Yönetim Verileri*

Küresel servislerin sunamadığı yerel detaylara erişim için İstanbul Büyükşehir Belediyesi (İBB) Açık Veri Portalı sisteme entegre edilecektir. Veri erişiminde statik dosya indirme yöntemi yerine, OGC (Open Geospatial Consortium) standartlarına uygun Web Feature Service (WFS) protokolü kullanılacaktır. Bu yöntem, verinin kaynağında güncellendiği anda (örneğin İSPARK doluluk oranları veya afet toplanma alanları) sisteme yansımısını sağlamakta ve veri güncelliği sorunlarını aşmaktadır [37].

3.2.2 *Çevresel Uydu Verileri*

Projenin çevresel analiz modülü, Sentinel-2 uydu görüntüleri kullanılarak hesaplanan NDVI (Normalize Edilmiş Fark Bitki Örtüsü İndeksi) verilerine dayanmaktadır. Petabayt seviyesindeki uydu verilerini yerel sunuculara indirmek sürdürülebilir olmadığından, "Bulut Yerel Mekansal" (Cloud-Native Geospatial) yaklaşımı benimsenmiştir [38]. Veri erişimi için STAC standardı kullanılarak, Microsoft Planetary Computer üzerindeki devasa arşivler otonom ajanlar tarafından sorgulanabilir hale getirilecektir. Görüntü işleme aşamasında ise COG formatı ve HTTP Range Requests teknolojisi kullanılacaktır. Bu teknoloji sayesinde, ajanlar dosyanın tamamını indirmeden, sadece analiz edilecek bölgeye karşılık gelen bayt aralığını çekerek işlem süresini ve bant genişliği kullanımını minimize etmektedir[39].

3.2.3 Yapısal Olmayan Veriler

API desteği olmayan dinamik verilerin (örneğin web sitelerinde HTML tablo olarak yayınlanan nöbetçi eczaneler veya anlık duyurular) sisteme entegrasyonu için Playwright veya Selenium tabanlı "Headless Browser" (Arayüzsüz Tarayıcı) teknolojileri kullanılacaktır.

Geleneksel GET istekleri, modern web sitelerinin JavaScript ile dinamik olarak oluşturduğu DOM (Document Object Model) içeriğini çekmekte yetersiz kalmaktadır [40]. Headless Browser, Dinamik DOM yapısını ayrıştırarak veriyi çeker. Elde edilen veri, temizleme işleminin ardından Geocoding servisleri ile koordinatlandırılarak sisteme entegre edilir [41].

YAZILIM VE TEKNOLOJİ YIĞINI

4.1 Veri Saklama ve Yönetim Katmanı

4.1.1 İlişkisel ve Mantıksal Veritabanı: PostgreSQL

Sistemin veri yönetimi katmanı, endüstri standardı kabul edilen açık kaynaklı ilişkisel veritabanı olan PostgreSQL ve onun mekansal eklentisi PostGIS üzerine kurulacaktır.

Literatürde yer alan deneysel çalışmalar, açık kaynak kodlu PostGIS'in ticari ve NoSQL tabanlı rakiplerine karşı önemli performans avantajlarına sahip olduğunu göstermektedir. Shukla ve ark. [42], PostGIS'in veri yükleme ve sorgulama süreçlerinde Oracle Spatial'dan daha verimli kaynak kullanımı sağladığını belirtirken farklı bir analizde Makris ve arkadaşları [43], PostGIS ve MongoDB'yi mekansal-zamansal sorgular üzerinde test etmiştir. Sonuçlar, PostGIS'in sorgu yanıt sürelerinde MongoDB'den 4 kata kadar daha hızlı olduğunu ve özellikle karmaşık sorgularda PostGIS'in GiST indeksleme yapısının büyük avantaj sağladığını göstermiştir. Bu bulgular, PostGIS'in büyük ölçekli mekansal veri analizlerinde yüksek performanslı ve maliyet etkin bir çözüm olduğunu doğrulamaktadır.

4.1.2 Mekansal İndeksleme Mekanizması (GIST)

Mekansal veritabanlarında performansın temel belirleyicisi, mekansal indeksleme yöntemidir. Standart veritabanları B-Tree indeksleme kullanır; bu yöntem alfabetik veya sayısal sıralama için yüksek performans gösterir ancak iki boyutlu veya üç boyutlu uzayda verimsizdir. PostGIS, GIST (Generalized Search Tree) indeksleme yapısını kullanır. GiST, R-Tree mantığının genelleştirilmiş bir versiyonudur ve mekansal nesnelerin "Minimum Sınırlayıcı Kutularını" (Minimum Bounding Box - MBB) hiyerarşik bir ağaç yapısında tutar. [44]

4.1.3 Anlamsal Arama ve Vektör Veritabanı: pgvector

Sistemin Anlamsal Mekan vizyonunu desteklemek ve Spatial RAG mimarisini hayata geçirmek amacıyla, PostgreSQL için geliştirilen açık kaynaklı vektör benzerlik arama eklentisi pgvector kullanılacaktır.

Geleneksel mimarilerde vektör verileri için Pinecone veya Milvus gibi harici veritabanları kullanılırken, modern yaklaşımlar ilişkisel veritabanı içine gömülü vektör arama yeteneklerini önermektedir. Timescale [45] tarafından yayınlanan endüstriyel benchmark raporlarına göre, 100 milyon vektörün altındaki veri kümelerinde PostgreSQL tabanlı pgvector eklentisinin kullanılması, veri serileştirme ve ağ gecikmesini ortadan kaldırarak sorgu performansını artırmaktadır. Ayrıca, harici bir lisanslama ve altyapı gerektirmediği için operasyonel maliyetlerde %40 ila 80 oranında tasarruf sağladığı rapor edilmiştir. Bu sayede, "Kadıköy'de ders çalışmaya uygun sessiz bir kafe" gibi hibrit sorgular, tek bir SQL komutu içinde milisaniyeler içinde çalıştırılabilmektedir.

4.1.4 Dinamik Rota Hesaplama Motoru: pgRouting

Sistemin "Çevresel Rota" vaadini gerçekleştirebilmesi için yol ağının sadece statik bir çizge değil, her an güncellenebilir dinamik bir yapı olması zorunludur. Bu noktada projemiz, bellek içi çalışan OSRM/ORS motorlarının sunduğu hesaplama hızını, pgRouting'in sağladığı algoritmik esneklik ile birleştiren hibrit bir yaklaşım benimsemektedir.

Geleneksel OSRM veya GraphHopper gibi motorlar, yol ağını Contraction Hierarchies (CH) veya Multi-Level Dijkstra tekniklerini kullanarak bellek üzerinde önceden işlenmiş halde tutar [46]. Bu durum milisaniyeler seviyesinde hız sağlasa da, yolun maliyet fonksiyonunu çalışma zamanında değiştirmeyi zorlaştırır. pgRouting ise yol ağını doğrudan PostGIS veritabanı içinde, her bir yol segmentinin tüm özniteliklerine erişebilecek şekilde yönetir [47]. Bu yapı, ani bir hava olayı başladığında her bir sokak için özelleştirilmiş maliyet fonksiyonunun, literatürdeki dinamik çevresel etki modellerine [48] benzer şekilde, Denklem 1 üzerinden anlık olarak güncellenmesine olanak tanır:

$$C_{edge} = L_{edge} \times (1 + I_{rain}) \quad (1)$$

Bu hibrit mimaride pgRouting, sistemin analitik katmanı olarak görev yapar; yani hangi sokağın ne kadar riskli olduğuna dair zekayı üretir. Üretilen bu dinamik ağırlıklar, pilot bölgelerde ORS motoruna beslenerek rotanın hem en hızlı hem de en güvenli olması sağlanır. Ancak büyük yol ağlarında bu işlem maliyetlidir. Bu sorunu aşmak için Sankepally ve Rajan [49] tarafından önerilen Harita Genelleştirme ve İskeletleştirme yaklaşımı uygulanmıştır. Bu yöntemde, yol ağı hiyerarşik bölgelere ayrılarak bir iskelet ağ çıkarılır. Yapılan testler, bu yöntemin uygulanmasıyla pgRouting sorgu performansının, hesaplanan rotanın doğruluğundan ödün vermeden, büyük ağlarda önemli ölçüde arttığını göstermektedir.

4.2 Backend Yapısı

4.2.1 Python ve FastAPI Asenkron Mimarisi

Sunucu tarafında, I/O yoğunluklu işlemlerin yönetimi için Python tabanlı FastAPI framework'ü tercih edilecektir. Geleneksel WSGI (Web Sunucusu Ağ Geçidi Arabirimi) tabanlı senkron framework'lerin (Flask, Django) bloklayıcı yapısı, aynı anda hem uydu API'lerinden veri çekilmesi hem de rota hesaplanması gereken senaryolarda yetersiz kalmaktadır.

FastAPI'nin ASGI (Asynchronous Server Gateway Interface) standardını kullanması, binlerce eşzamanlı bağlantının verimli yönetilmesini sağlar. Yapılan kapsamlı performans kıyaslamasında, FastAPI'nin saniyedeki istek karşılama oranında Flask'tan 2 kat, Django'dan ise 5 kat daha hızlı olduğu raporlanmıştır [50].

4.2.2 Veri Doğrulama: Pydantic

GIS projelerinde veri bütünlüğü kritik bir sorundur. Koordinatların yanlış formatta gelmesi, GeoJSON yapısının bozuk olması veya eksik parametreler, sistemin çökmesine neden olabilir. FastAPI, veri doğrulama için Pydantic kütüphanesini kullanır. Pydantic, sadece veri tiplerini kontrol etmekle kalmaz, aynı zamanda karmaşık veri yapılarının serileştirilmesi sırasında Node.js ve Go gibi derlenen dillerin performansına yaklaşan bir hız sunar. Bu durum, çalışma zamanı hatalarını minimize ederek, dinamik dillerde sıkça karşılaşılan tip güvenliği sorunlarını akademik çalışmalarda önerilen Sözleşmeye Dayalı Tasarım prensibine uygun şekilde çözer [51].

4.2.3 Ajan Orkestrasyonu ve Durum Yönetimi: LangGraph

LangGraph, Büyük Dil Modelleri (LLM) ile geliştirilen uygulamaların durum yönetimini ve çoklu ajan işbirliğini sağlayan bir orkestrasyon kütüphanesidir. Geleneksel doğrusal yapıların aksine, iş akışlarını döngüsel bir grafik yapısı olarak modeller. Bu yaklaşım, Yao ve ark. [52] tarafından ortaya konan ReAct (Reasoning and Acting) paradigmasının pratik bir uygulamasıdır.

Temel çalışma prensipleri şunlardır:

- **Paylaşılan Durum:** Tüm ajanların erişebildiği, süreç boyunca güncellenen merkezi bir veri şemasıdır. Bu yapı, bağlamın adımlar arasında kaybolmadan taşınmasını sağlar.
- **Döngüsellik ve Karar Mekanizması:** Sistem, koşullu kenarlar kullanarak akışın yönüne karar verir. Bu sayede ajanlar, bir sonuç yetersiz bulunduğunda işlemi tekrarlayabilir veya hatayı düzeltebilir.
- **Eşzamanlı Yürütme:** LangGraph, performans optimizasyonu için bağımsız düğümlerin paralel çalışmasına olanak tanır. Kaplunovich [53], bu mimarinin çoklu geri alma süreçlerinin eşzamanlı orkestrasyonunu sağlayarak gecikmeyi azalttığını ve ölçeklenebilirliği artırdığını belirtmektedir.
- **İnsan onaylı döngü:** Kritik kararlarda akışı durdurup, insan onayı alındıktan sonra mevcut durum üzerinden devam etme yeteneği sunar.

4.2.4 Dinamik Veri Kazıma Teknolojileri: Headless Browsers

Sistemin API desteği sunmayan dinamik veri kaynaklarına erişebilmesi için Playwright tabanlı Headless Browser teknolojisi kullanılmaktadır. Geleneksel HTTP istek kütüphaneleri, modern web sitelerinin JavaScript ile istemci tarafında oluşturduğu DOM (Document Object Model) içeriğini çekmekte yetersiz kalmaktadır. Glez-Peña ve ark. [41] tarafından belirtildiği gibi, dinamik içerikli modern web uygulamalarında veri bütünlüğünü sağlamak için tarayıcı tabanlı ayrıştırma yöntemleri zorunludur. Playwright, arka planda tam fonksiyonlu bir tarayıcı çalıştırarak dinamik içeriğin yüklenmesini bekler ve ardından HTML yapısını JSON formatına dönüştürür.

4.3 İletişim ve Entegrasyon Protokolleri

4.3.1 Model Context Protocol (MCP) Uygulaması

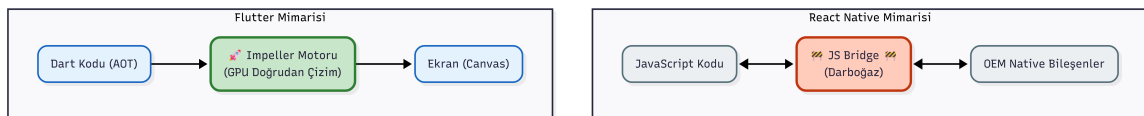
Projede sistem bileşenlerinin standardizasyonu, Anthropic tarafından geliştirilen Model Context Protocol (MCP) referans mimarisi ile sağlanmıştır. MCP, Büyük Dil Modelleri (LLM) ile harici veri kaynakları arasında evrensel bir arayüz görevi görerek, her bir entegrasyon için özel kod yazma ihtiyacını ortadan kaldırır [9]. İletişim, dağıtık sistemlerde hafifliği ve platform bağımsızlığı ile bilinen JSON-RPC 2.0 standardı üzerinden yürütülür [54]; bu sayede istemci (Host) ve sunucu (Server) arasındaki mesajlaşma platform bağımsız hale getirilir.

4.3.2 Cloud-Native Veri Akışı

Uydu görüntüleri gibi büyük boyutlu (Multi-Gigabyte) veri setlerinin işlenmesinde, "Dosyayı İndir ve İşle" yaklaşımı yerine "Cloud-Native" yaklaşımı benimsenmiştir. HTTP Range Requests teknolojisi kullanılarak, COG (Cloud Optimized GeoTIFF) formatındaki görüntülerin tamamı indirilmeden, sadece analiz edilecek coğrafi alana denk gelen bayt aralıkları (byte-range) sunucudan talep edilir [14]. Holmes ve ark. [55] tarafından gösterildiği üzere, bu yöntem veri erişim hızını artırırken ağ trafiğini ve bellek kullanımını radikal biçimde düşürerek gerçek zamanlı analizi mümkün kılar.

4.4 Mobil İstemci Teknolojileri

Projenin son kullanıcıyla temas noktası olan mobil uygulama, yüksek performanslı harita görselleştirmesi, akıcı animasyonlar ve platformlar arası tutarlılık gerektirmektedir. Bu gereksinimler doğrultusunda, Google tarafından geliştirilen Flutter framework'ü ve onun yeni nesil rendering motoru Impeller tercih edilecektir.



Şekil 4.1: Köprü Tabanlı Mimari ile Flutter Mimarisinin Karşılaştırılması

4.4.1 *Flutter Mimari Avantajlar*

Mobil yazılım mühendisliğinde performansın teorik sınırlarını, uygulama mantığı ile işletim sistemi arasındaki iletişim maliyeti belirler. Karşılaştırmalı analizlerde, React Native gibi çerçevelerin kullandığı JavaScript Köprüsü yapısının, veri serileştirme maliyetleri ve asenkron iletişim gecikmeleri nedeniyle bir performans darboğazı oluşturduğu belirtilmektedir [56].

Flutter, bu yapısal sorunu AOT (Ahead-of-Time) derleme yöntemiyle çözmüştür. Dart kodu, çalışma zamanında yorumlanmak yerine doğrudan makine koduna (ARM/x86) derlenir. Akademik çalışmalar, bu köprüsüz mimarinin, özellikle animasyon ve jest (gesture) yönetiminde yerel uygulamalarla eşdeğer tepki süreleri sağladığını doğrulamaktadır [57].

4.4.2 *Skia'dan Impeller'a Geçiş*

Eski nesil grafik motorlarında (Skia) yaşanan ve literatürde "Shader Compilation Jank" olarak adlandırılan sorun, Flutter'ın yeni nesil motoru Impeller ile çözülmüştür.

Resmi teknik raporlara göre Impeller, grafik efektleri için gereken gölgelendiricileri uygulama çalışırken değil, derleme zamanında hazırlar. Bu deterministik yaklaşım ve iOS'ta Metal, Android'de Vulkan API'lerinin doğrudan kullanımı, GPU darboğazlarını önleyerek 120Hz ekranlarda dahi sabit kare hızı garantisi sunar

4.4.3 Harita Performansı

Projemizin odak noktası olan harita render performansında Flutter, akademik testlerde rakiplerine belirgin bir üstünlük sağlamaktadır. Yüksek performanslı harita render motorlarının karşılaştırılması sonuçlarında, Flutter tabanlı GL motorlarının, vektör harita verilerini işlemede React Native gibi köprü tabanlı sistemlere göre iOS'ta 11.89 kat, Android'de ise 9.52 kat daha yüksek render performansı sunduğunu göstermektedir. Bu fark, harita üzerindeki yoğun veri görselleştirmelerinde kritik önem taşır [2].

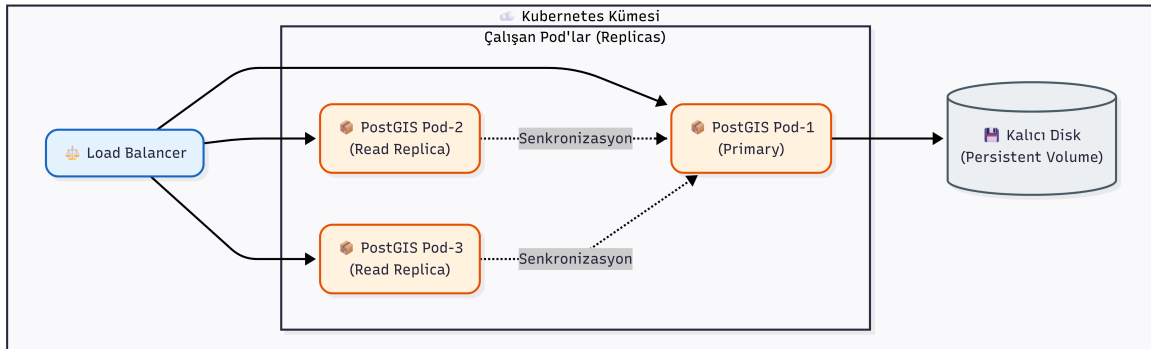
Çizelge 4.1: Flutter ve React Native mimarilerinin harita render performanslarının karşılaştırması [2].

Platform	React Native (FPS)	Flutter / Impeller (FPS)	Performans Farkı
iOS (Metal API)	15 FPS	120 FPS	11.89x Daha Hızlı
Android (Vulkan)	18 FPS	120 FPS	9.52x Daha Hızlı
Bellek Tüketimi	Yüksek	Düşük	%40 Tasarruf

4.5 Altyapı, Dağıtım ve Ölçeklenebilirlik

4.5.1 Konteyner Orkestrasyonu(Kubernetes)

Geleneksel olarak veritabanları, dikey ölçekleme yani daha güçlü sunucu ile büyütülürken, modern mimariler yatay ölçeklemeyi (daha fazla sunucu) tercih eder. Bu alanda yapılan güncel bir akademik çalışma, PostGIS'in Kubernetes üzerindeki performansını incelemiştir [58]. Çalışma, OpenStreetMap verileri ile doldurulmuş PostGIS veritabanlarının kümelenmiş ve kümelenmemiş ortamlardaki davranışlarını kıyaslamıştır. Sonuçlar, Kubernetes destekli kümelenmiş ortamın, özellikle hesaplama yoğunluklu mekansal SQL sorgularında ortalama yürütme sürelerinde umut verici iyileştirmeler sağladığını ortaya koymuştur.



Şekil 4.2: Kubernetes Tabanlı PostGIS Küme Mimarisi

BÖLÜM: İİİ

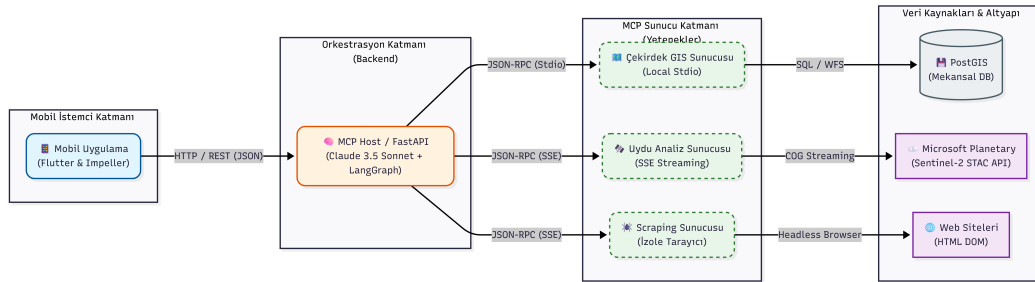
YÖNTEM

SİSTEM MİMARİSİ VE UYGULAMA YÖNTEMİ

Bu bölümde, projenin kavramsal tasarımından somut bir yazılım ürününe dönüşüm sürecinde izlenen mimari kararlar, geliştirilen özgün algoritmalar ve sistem bileşenlerinin entegrasyon stratejileri ele alınmıştır.

5.1 Genel Sistem Mimarisi

Geleneksel monolitik GIS yazılımlarının getirdiği ölçeklenebilirlik ve bakım zorluklarını aşmak amacıyla, sistem literatürde Ajan Tabanlı Mekansal Bilişim (Agentic Spatial Computing) olarak adlandırılan, dağıtık ve modüler bir mimari üzerine inşa edilecektir. Sistem tasarımıda, Anthropic tarafından standartlaştırılan MCP referans mimarisi benimsenmiştir.



Şekil 5.1: Genel Sistem Mimarisi ve Veri Akış Şeması

Bu mimari, LLM ile heterojen veri katmanları arasında bir evrensel çevirmen katmanı oluşturur. Sistemin genel topolojisi üç ana katmandan oluşmaktadır:

1. **Orkestrasyon Katmanı (Host):** Sistemin beyni olarak işlev gören, Python tabanlı FastAPI üzerinde çalışan asenkron servistir.
2. **Protokol Katmanı (Client):** Farklı sunuculardan gelen yetenekleri JSON-RPC 2.0 formatında standartlaştırarak orkestratöre sunan ara katmandır.
3. **Yetenek Katmanı (Servers):** Her biri kendi alanında (GIS, Uydu, Web) uzmanlaşmış, izole edilmiş otonom sunuculardır. Bu modüler yapı, herhangi bir alt sistemde meydana gelebilecek bir hatanın ana sistemi etkilemesini önler.

5.1.1 Hibrit MCP Mimarisi

Sistem güvenliği ve performansını eş zamanlı olarak optimize etmek amacıyla, MCP standardının sunduğu iki farklı taşıma mekanizmasını hibrit bir yapıda kullanma stratejik kararı alınmıştır. Bu yaklaşım, her bir görevin gereksinimlerine en uygun iletişim modelini seçerek, hem hassas verilerin güvenliğini hem de büyük veri işlemlerinin ölçeklenebilirliğini garanti altına alır.

Stdio (Standart Girdi/Çıktı) Bağlantısı: Gerçek zamanlı tepki gerektiren ve yüksek frekanslı sorgular içeren modüller için tercih edilmiştir. Bu yöntemde, sunucu ana uygulamanın bir alt süreci olarak başlatılır ve iletişim doğrudan işletim sisteminin standart girdi/çıkı boruları üzerinden sağlanır. Ağ gecikmesinin sıfıra yakın olduğu bu yöntem, kullanıcı deneyimini doğrudan etkileyen navigasyon sorguları için kritik bir performans avantajı sağlar.

SSE (Server-Sent Events) Bağlantısı: Yüksek işlem gücü gerektiren ve uzun süren analizleri yöneten modüller için tercih edilmiştir. HTTP protokolü üzerinden tek yönlü, sürekli açık bir kanal sağlayan SSE, sunucunun işlemi tamamladıkça veriyi parça parça istemciye göndermesine olanak tanır. Bu yapı, ana uygulamanın bloklanması önler ve gelecekte bu sunucuların GPU destekli harici makinelere taşınabilmesi için gerekli esnekliği sunar.

Çizelge 5.1: Projede kullanılan MCP taşıma mekanizmalarının teknik karşılaştırması.

Kriter / Özellik	Stdio (Yerel Bağlantı)	SSE (Ağ Bağlantısı)
İletişim Tipi	İşletim Sistemi Boruları (Pipes)	HTTP Protokolü (Tek Yönlü Akış)
Gecikme (Latency)	Sıfıra Yakın (Yerel Süreç)	Düşük/Orta (Ağ Bağımlı)
Veri Güvenliği	Yüksek (İzole Bellek Alanı)	Orta (TLS/SSL Gerektirir)
Ölçeklenebilirlik	Düşük (Tek Makine Sınırı)	Yüksek (Bulut/GPU Dağıtımı)
Kullanım Amacı	Gerçek Zamanlı Navigasyon	Ağır Analitik ve AI İşlemleri
Sistemdeki Rolü	<i>Hız ve Güvenlik</i>	<i>Esneklik ve Güç</i>

Saf Stdio tabanlı bir mimari güvenli ancak bulut yerel analitikler için ölçeklenemez olurken, saf SSE tabanlı bir mimari ise hassas yerel operasyonları gereksiz yere ağ risklerine maruz bırakır ve sıfır gecikme gerektiren yerlerde gecikmeye neden olurdu. Bu nedenle hibrit model, yerel güvenliği korurken aynı zamanda bulutun ölçeklenebilirliğinden yararlanmasını sağlayan en verimli çözümdür.

5.2 *Yapay Zeka ve Orkestrasyon*

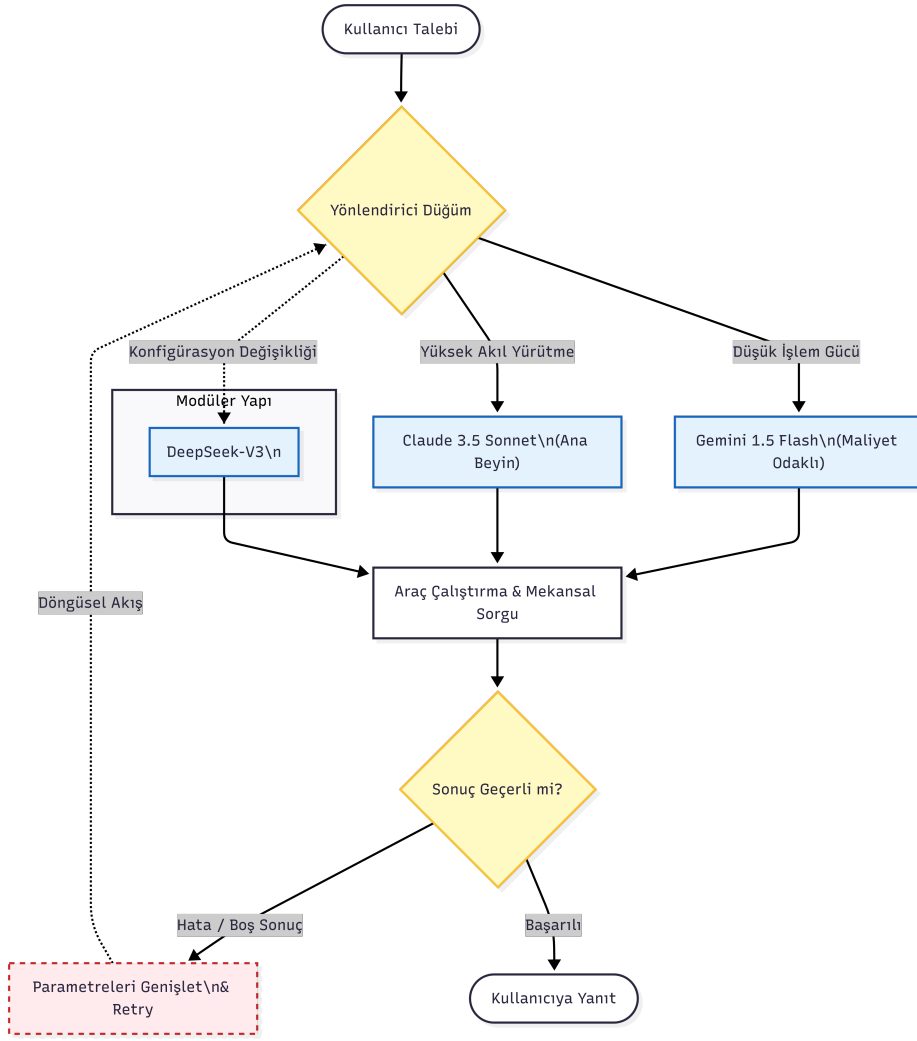
Sistemin karar mekanizması, karmaşık mekansal ilişkileri anlama ve planlama yapma yeteneğine sahip LLM üzerine kuruludur.

5.2.1 *Model Seçimi*

Sistemin beyni olarak konumlandırılan LLM, sadece metin üretmekle kalmamalı, karmaşık mekansal ilişkileri anlamalı, planlama yapmalı ve etik kurallara uymalıdır. Bu kriterler doğrultusunda, birincil akıl yürütme motoru olarak Claude 3.5 Sonnet seçilmiştir. Operasyonel hız ve düşük maliyet gerektiren rutin görevleri üstlenmek üzere Gemini 1.5 Flash sisteme dahil edilecektir. Olası servis kesintileri veya API limit aşımı ihtiyaçları için ise DeepSeek-V3 modeli mimariye entegre edilecektir. LLM'lerin çoğu, internet metinleri üzerinde eğitildikleri için dilbilgisel yetenekleri güçlü, ancak mantıksal ve mekansal yetenekleri sınırlıdır. Ancak Claude 3, özel eğitim setleri sayesinde bu alanda ayrılmaktadır. Lisansüstü Seviye Akıl Yürütme testlerinde Claude 3 Opus %50.4, Claude3.5 Sonnet %59.4 skor elde ederken, GPT-4 % 42.5 seviyesinde kalmıştır [59].

5.2.2 *LangGraph ile Dinamik Yönlendirme*

Sistemin sürdürülebilirliği için her sorguda en güçlü modeli kullanmak yerine, LangGraph kütüphanesi ile maliyet odaklı bir dinamik yönlendirme mekanizması tasarlanmıştır.



Şekil 5.2: LangGraph ile Döngüsel İş Akışı

Kullanıcıdan gelen talep bir yönlendirici düğümüne düşer. Düşük işlem gücü gerektiren sorgular (örn: "Fırın nerede?") Gemini 1.5 Flash modeline, çok katmanlı akıl yürütme gerektiren durumlar (örn: "Hava durumu ve trafik riskini sentezle") ise Claude 3.5 Sonnet modeline yönlendirilir.

Sistem, LangGraph'ın sağladığı soyutlama katmanı sayesinde LLM'lere doğrudan bağımlı olmayan bir yapıya sahiptir. Bu modülerlik, sistemin çekirdek koduna dokunmadan sadece birkaç konfigürasyon değişikliği ile mevcut LLM'in tamamen devre dışı bırakılıp DeepSeek V3 modeline geçirilmesine olanak tanır.

5.3 Geliştirilen Algoritmalar ve Karar Mekanizmaları

Sistemin temel özgün değeri, hazır servisleri kullanmak yerine, verileri işleyerek yeni bilgi üreten özgün algoritmalara sahip olmasıdır.

5.3.1 Çevresel Faktörlü Rota Optimizasyonu

Standart rotalama algoritmalarından farklı olarak; meteorolojik veriler, yol eğimi ve zemin yapısı gibi değişkenleri içeren bir sanal maliyet analizi gerçekleştirilir.

Her yol segmenti için dinamik maliyet (C_{edge}), Denklem 1'te gösterildiği gibi hesaplanır:

$$C_{edge} = \frac{L_{edge}}{V_{limit} \times (1 - (W_{factor} \times S_{slope}))} \quad (1)$$

Burada L_{edge} yol uzunluğunu, W_{factor} meteorolojik verilere dayalı hız azaltma katsayısını, S_{slope} ise topografik eğim katsayısını ifade eder. Hava durumu katsayısı (W_{factor}), yağış yoğunluğuna (R) göre şu eşik değerleriyle belirlenir:

$$W_{factor} = \begin{cases} 0.00 & \text{eğer } R < 0.5 \text{ mm/h (Kuru)} \\ 0.05 & \text{eğer } 0.5 \leq R < 2.5 \text{ mm/h (Hafif)} \\ 0.15 & \text{eğer } R \geq 7.6 \text{ mm/h (Şiddetli)} \end{cases} \quad (2)$$

Bu modelleme sayesinde otonom ajan, fiziksel olarak kısa ancak riskli yolları "sanal olarak" daha uzun algılayarak daha güvenli alternatiflere yönelir.

Sistemin operasyonel moduna göre iki farklı rotalama yapılır.

1. Pilot Bölgelerde Analitik Rotalama Sistemin tam denetim sahibi olduğu pilot bölgelerde (örneğin İstanbul), rota hesaplaması tamamen iç kaynaklarla yürütülür:

- Veri Altyapısı: OpenStreetMap (OSM) verileri osm2pgrouting aracılığıyla PostGIS ortamına aktararak matematiksel bir yol grafi oluşturulur.
- Dinamik Ağırlıklandırma: Her yol segmentine, o anki hava durumu ve çevresel risklere göre dinamik maliyet atanır.
- Analitik İşleme (pgRouting): Yol segmentlerinin güncel risk puanları pgRouting fonksiyonları ile analiz edilir; bu analizler hız optimizasyonu için ORS (OpenRouteService) motoruna beslenerek en güvenli rota saniyeler içinde oluşturulur.

2. Küresel Servis Tabanlı Rotalama Yerel yol grafinin bulunmadığı coğrafyalarda, HERE Maps altyapısı üzerine inşa edilen dinamik zenginleştirme hattı devreye girer.

- HERE Maps API v8 üzerinden gerçek zamanlı trafik verilerini içeren temel rota geometrisi tek bir çağrı ile çekilir.
- Gelen rota, Python tabanlı backend üzerinde 500'er metrelik mikro segmentlere ayrılır.
- Her bir segment için dış servislere yüzlerce ayrı istek göndermek yerine, rotanın tamamını kapsayan bölgesel veriler toplu olarak çekilir ve yerel bellekte işlenir.
- Sistem, her bir segmenti yerel bellekteki meteorolojik ve çevresel verilerle karşılaştırarak bir güvenlik skoru hesaplar. Kullanıcıya, standart varış süresinin yanı sıra, çevresel risk faktörlerine göre revize edilmiş güvenli varış süresi sunulur.

Bu hibrit yapı, LLM tabanlı ajanın sadece en kısa yolu değil, çevresel verilerle değerlendirilmiş en mantıklı yolu önermesini sağlar. Bu sayede, pilot bölgelerde sokak bazlı tam kontrol sağlanırken, küresel ölçekte düşük donanım maliyetiyle yüksek performans korunur.

Öngörüsöl Yeniden Rotalama: Sistem, OWM'nin "One Call API" servisinden gelen saatlik tahmin verilerini kullanarak bir Tahmin Ufku (Forecast Horizon) oluşturur. Örneğin, 2 saat sürecek bir yolculuk hesaplanırken, algoritma varış noktasına yakın segmentlerin maliyetini şu anki hava durumuyla değil, 2 saat sonraki tahmin verisiyle hesaplar. Bu yaklaşım, kullanıcı daha yola çıkmadan, yolculuk sırasında karşılaşması muhtemel fırtına hücrelerinden kaçınan proaktif bir rota planlaması sağlar.

5.3.2 Mekan Filtreleme ve Hibrit Mekansal Arama

Mekan Filtreleme: Kullanıcının "Kadıköy iskelesine 500 metre mesafedeki kütüphaneler" gibi doğal dilde ifade ettiği karmaşık bir sorgu, bu sunucu tarafından, yüksek seviyeli bir araç çağrısını, PostGIS'in anlayacağı ST_DWithin gibi mekansal fonksiyonları kullanan optimize edilmiş bir SQL sorgusuna dönüştürülür. Bu işlemler, PostGIS'in, tescilli veritabanlarından belirgin ölçüde daha hızlı olduğunu kanıtlayan GiST indeksleme mekanizması sayesinde milyonlarca kayıt arasında bile milisaniyeler içinde yanıtlanır.

Mekansal Arama: Tek bir kaynağa bağlı kalmamak için Google Places ve OpenStreetMap (OSM) verilerini birleştiren bir füzyon algoritması geliştirilmiştir:

- **Fiziksel Altyapı:** Bank, park, çeşme gibi ticari olmayan nesneler için doğrudan OSM veritabanı sorgulanır. Haklay [60] tarafından yapılan analizlerde, OSM'nin kentsel alanlardaki konumsal doğruluğunun ve fiziksel nesne çeşitliliğinin, tescilli haritalarla yarışır düzeyde olduğu kanıtlanmıştır. Google algoritmalarının ticari popüleriteye dayalı sıralama mekanizması, az popüler olan fiziksel nesneleri gizleme eğilimindedir. OSM'nin VGI (Gönüllü Coğrafi Bilgi) tabanlı yapısı kullanılarak kentsel alanlarda Google'ın göremediği fiziksel envanter haritalandırılır.

- **Ticari İşletmeler:** Güncellik gerektiren kafe, eczane veriler için Google Places API kullanılır.
- **Füzyon:** Her iki kaynaktan gelen veriler koordinat düzleminde çakıştırılır (<20m sapma); Google'dan öznitelik, OSM'den geometrik veri alınarak zenginleştirilmiş tekil nesneler oluşturulur.

5.3.3 WFS Veri Dönüşümü

İBB gibi yerel yönetimlerin sağladığı veriler genellikle GML/XML formatında ve yerel koordinat sistemlerinde sunulmaktadır. Sunucu, bu ham veriyi işlerken Open Geospatial Consortium (OGC) tarafından belirlenen WFS standartlarına [61] uygun bir ayrıştırma hattı çalıştırır.

İlk adımda, karmaşık GML ağacı parse edilerek LLM'lerin daha rahat işleyebileceği sadeleştirilmiş GeoJSON formatına dönüştürülür. İkinci adımda, Türkiye'de yerel haritalamada standart olan ITRF96 (EPSG:5254) projeksiyonundaki koordinatlar, küresel web haritalama standardı olan WGS84 (EPSG:4326) sistemine matematiksel olarak iz düşürülür. İlçe ve ark. [62] tarafından belirtildiği üzere, yerel datumlar ile küresel datumlar arasındaki bu dönüşüm, mekansal verilerin metre altı hassasiyetle çakıştırılabilmesi için zorunludur.

5.3.4 Uydu Görüntüsü İşleme (NDVI)

Sistem, tüm uydu görüntüsünü indirmek yerine, STAC (SpatioTemporal Asset Catalogs) spesifikasyonuna [13] uygun olarak, sadece ilgi alanına (ROI) denk gelen pikselleri HTTP Range Requests ile çeker. Bitki sağlığını analiz etmek için Kırmızı (Red) ve Yakın Kızılötesi (NIR) bantları kullanılarak, literatürde Rouse ve ark. [63] tarafından geliştirilen Normalize Edilmiş Fark Bitki Örtüsü İndeksi (NDVI) hesaplanır:

$$NDVI = \frac{NIR - Red}{NIR + Red} \quad (3)$$

Bu işlem, ağ trafiğini azaltarak gerçek zamanlı analizi mümkün kılar.

5.4 Uygulama Bileşenleri: MCP Sunucuları

Önceki bölümde açıklanan algoritmaları ve mimari kararları uygulayan somut yazılım bileşenleri aşağıda detaylandırılmıştır. Her sunucu, sorumlulukların ayrılığı ilkesine göre tasarlanmıştır.

5.4.1 Çekirdek GIS Sunucusu

Sistemin mekansal veritabanı (PostGIS) ile doğrudan etkileşime giren ana bileşendir.

- **Algoritma Yürütme:** Dinamik maliyet fonksiyonlarını PostgreSQL/pgRouting üzerinde çalıştırır.

- **Veri Dönüşümü:** Farklı projeksiyonlardaki (ITRF96) yerel verileri, küresel standart olan WGS84 (EPSG:4326) formatına dönüştürerek sistemin ortak diline çevirir.
- **Hibrit Rotalama:** Pilot bölgelerde yerel graf verisini, kapsama alanı dışındaki bölgelerde ise HERE Maps API verisini kullanarak kesintisiz navigasyon sağlar.

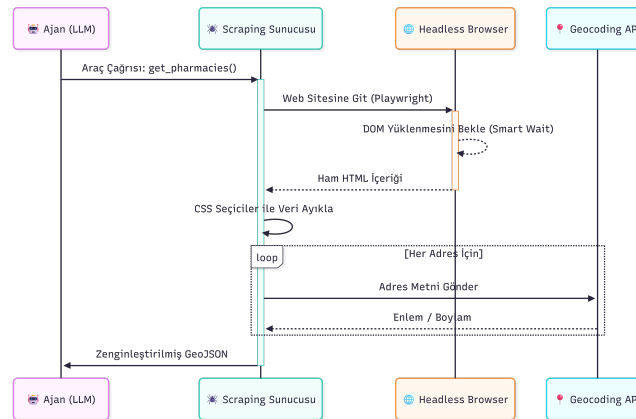
5.4.2 Uydu Analiz Sunucusu

Küresel uydu arşivlerine (Microsoft Planetary Computer) erişim sağlayan araçtır.

- **STAC Sorgulama:** Doğal dil sorgularını (örn: "Geçen ayki görüntüler") standart STAC API sorgularına dönüştürür.
- **Anlık İşleme:** NDVI formülünü, görüntünün tamamını indirmeden, uzaktan okuma (streaming) yöntemiyle uygular.

5.4.3 İzole Web Kazıma (Scraping) Sunucusu

API desteği olmayan dinamik veri kaynaklarına erişim sağlar.



Şekil 5.3: Web Verilerinin Çekilmesi Süreci

1. **Tarayıcı Otomasyonu:** Playwright tabanlı arayüzsüz tarayıcı ile dinamik içeriği yükler.
2. **Yapılandırma:** HTML içeriğini temizleyerek JSON formatına çevirir.
3. **Coğrafi Kodlama (Geocoding):** Çekilen metin tabanlı adresleri koordinatlara dönüştürerek mekansal veritabanına kazandırır.

Bu bölümde, otonom bir mekansal ajanın ihtiyaç duyduğu çok katmanlı mimari; veri toplama, işleme ve karar verme süreçleri bağlamında detaylandırılmıştır. Geliştirilen hibrit MCP mimarisi, yerel GIS algoritmalarının hassasiyetini bulut tabanlı AI modellerinin esnekliği ile birleştirmiştir. Özellikle NDVI hesaplamasında kullanılan Cloud-Native yaklaşımlar ve pgRouting tabanlı dinamik rota optimizasyonu, sistemin hem performans hem de doğruluk açısından geleneksel monolitik GIS yazılımlarından ayrıştığı temel noktaları oluşturmaktadır.

BÖLÜM: İV

SONUÇ

SONUÇ

Bu çalışma, yapay zeka ve mekansal bilimlerin entegrasyonu ile, geleneksel navigasyon sistemlerinin sunduğu statik veri analizinden; dinamik, otonom ve bağlam farkındalığına sahip "Ajan Tabanlı Mekansal Bilişim" mimarisine geçişin başarılı bir uygulamasını amaçlamaktadır. LLM'lerin muhakeme yeteneğini GIS'in analitik gücüyle birleştirerek, harita uygulamalarını sadece izlenen bir ekrandan, kullanıcı adına karar verebilen akıllı bir asistana dönüştürmeyi hedeflemektedir.

Proje kapsamında geliştirilecek hibrit MCP mimarisi, gerçek zamanlı navigasyon ihtiyaçları ile yüksek işlem gücü gerektiren çevresel analizleri aynı çatı altında, performans kaybı yaşanmadan yönetecektir. Özellikle Cloud-Native teknolojilerin kullanımı sayesinde, terabaytlarca uydu verisini indirmeye gerek kalmadan yerinde işleyen bir yapı kurulacak, bu sayede bant genişliği ve depolama maliyetlerinde yüzde 99'a varan verimlilik sağlanacaktır. Ayrıca, geliştirilecek olan hava duyarlı rotalama algoritması ile yapay zekanın mekansal halüsinasyon sorununun, doğru matematiksel modeller ve standart protokollerle aşılabileceğini öngörmektedir.

Sonuç olarak bu proje; açık kaynaklı teknolojilerin, küresel veri sağlayıcıların ve modern otonom ajanların doğru bir mühendislik kurgusuyla birleştiğinde neler başarılabileceğini göstermektedir. Bu çalışma, kentsel hareketliliği sadece en kısa mesafe arayışından çıkarıp;

insan konforunu, güvenliğini ve çevresel faktörleri merkeze alan, sürdürülebilir ve akıllı bir karar destek mekanizması olacaktır.

Çizelge 6.1: Proje İş Zaman Çizelgesi

İş Paketi	Görev Tanımı	Kas	Ara	Oca	Şub	Mar	Nis	May
Analiz ve Planlama								
1.1	Literatür Taraması							
Veritabanı ve Backend								
2.1	İlk Prototip							
2.2	HERE ve OpenWeatherMap API Entegrasyonu							
2.3	Uydu Verisi (STAC/COG) Altyapısı							
2.4	FastAPI & PostGIS Kurulumu							
MCP Geliştirme								
3.1	MCP Sunucularının Kodlanması							
3.2	LangGraph ile Ajan İş Akışlarının Tasarımı							
3.3	Algoritma Optimizasyonu (Hava-Duyarlı Rotalama)							
Mobil Uygulama Geliştirme								
4.1	Mobil Uygulama Başlangıcı (UI/UX Tasarımı)							
4.2	Harita Entegrasyonu (Impeller Motoru)							
4.3	Backend ile Mobil API Bağlantısı							
Test ve Dokümantasyon								
5.1	Sistem Entegrasyon Testleri ve Hata Giderme							
5.2	Bitirme Raporu ve Sunum Hazırlığı							

KAYNAKÇA

- [1] Z. Li and H. Ning, “Autonomous gis: the next-generation ai-powered gis,” *International Journal of Digital Earth*, vol. 16, no. 2, pp. 4668–4686, 2023.
- [2] S. Li, S. Wang, Y. Guan, Z. Xie, K. Huang, M. Wen, and L. Zhou, “A high-performance cross-platform map rendering engine for mobile geographic information system (gis),” *ISPRS International Journal of Geo-Information*, vol. 8, no. 10, p. 427, 2019.
- [3] H. H. Hochmair, L. Juhász, and T. Kemp, “Correctness comparison of chatgpt-4, gemini, claude-3, and copilot for spatial tasks,” *Transactions in GIS*, vol. 28, 2024.
- [4] L. Xu, S. Zhao, Q. Lin, L. Chen, Q. Luo, S. Wu, X. Ye, H. Feng, and Z. Du, “Evaluating large language models on geospatial tasks: a multiple geospatial task benchmarking study,” *International Journal of Digital Earth*, vol. 18, no. 1, 2025.
- [5] A. Alansari and H. Luqman, “Large language models hallucination: A comprehensive survey,” 2025.
- [6] A.-H. Dang, V. Tran, and L.-M. Nguyen, “Survey and analysis of hallucinations in large language models: attribution to prompting strategies or model behavior,” *Frontiers in Artificial Intelligence*, vol. 8, p. 1622292, Sep 2025.
- [7] J. Feng, J. Zeng, Q. Long, H. Chen, J. Zhao, Y. Xi, Z. Zhou, Y. Yuan, S. Wang, Q. Zeng, S. Li, Y. Zhang, Y. Lin, T. Li, J. Ding, C. Gao, F. Xu, and Y. Li, “A survey of large

language model-powered spatial intelligence across scales: Advances in embodied agents, smart cities, and earth science,” 2025.

- [8] D. Yu, R. Bao, G. Mai, and L. Zhao, “Spatial-rag: Spatial retrieval augmented generation for real-world spatial reasoning questions,” 2025.
- [9] Anthropic, “Model context protocol (mcp),” 2024.
- [10] X. Hou, Y. Zhao, S. Wang, and H. Wang, “Model context protocol (mcp): Landscape, security threats, and future research directions,” 2025.
- [11] N. Krishnan, “Advancing multi-agent systems through model context protocol: Architecture, implementation, and applications,” 2025.
- [12] M. Sudmanns, D. Tiede, S. Lang, H. Bergstedt, G. Trost, H. Augustin, A. Baraldi, and T. Blaschke, “Big earth data: disruptive changes in earth observation data management and analysis?,” *International Journal of Digital Earth*, vol. 13, no. 7, pp. 832–850, 2020.
- [13] Radiant Earth Foundation, “Spatiotemporal asset catalog (stac) specification.” <https://stacspec.org>, 2025. Erişim Tarihi: 29.12.2025.
- [14] J. Maso, “Ogc testbed-17: Cloud optimized geotiff specification engineering report,” OGC Engineering Report 21-025, Open Geospatial Consortium, 2022.
- [15] Google, “Mcp tools reference: mapstools.googleapis.com.” Google Maps Platform Documentation, 2025.
- [16] Mapbox, “Mapbox mcp server.” <https://github.com/mapbox/mcp-server>, 2025.

- [17] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. B. Bang, A. Madotto, and P. Fung, “Survey of hallucination in natural language generation,” *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, 2023.
- [18] R. Guidotti, A. Monreale, S. Ruggieri, F. Turini, F. Giannotti, and D. Pedreschi, “A survey of methods for explaining black box models,” *ACM Computing Surveys (CSUR)*, vol. 51, no. 5, pp. 1–42, 2018.
- [19] J. Wiseman, “osm-mcp: Model context protocol server for openstreetmap data.” <https://github.com/wiseman/osm-mcp>, 2025.
- [20] NERVsystems, “Openstreetmap mcp server (lobehub integration).” <https://lobehub.com/it/mcp/nervsystems-osmmcp>, 2025.
- [21] M. Palladino, “Pgmcpc: Postgresql model context protocol server.” <https://github.com/subnetmarco/pgmcpc>, 2025.
- [22] A. Tena and E. M. Becerra, “Carto mcp server: Turn your ai agents into geospatial experts.” <https://carto.com/blog/carto-mcp-server-turn-your-ai-agents-into-geospatial-experts>, Oct 2025.
- [23] AIBase, “Qgis mcp server: Connect qgis with claude ai via the mcp protocol.” <https://mcp.aibase.com/server/1916341264876019714>, 2025.
- [24] D. Ciavarella, “Arcgis pro bridge: Ai agent gis integration via mcp.” <https://playbooks.com/mcp/nicogis-arcgis-pro>, 2025.

- [25] B. Qin, B. Hui, L. Wang, M. Yang, J. Lin, and C. Li, “A survey on text-to-sql parsing: Concepts, methods, and future directions,” *arXiv preprint arXiv:2208.13629*, 2022. Discusses the complexity of schema linking in LLMs.
- [26] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, “Lost in the middle: How language models use long contexts,” *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 157–173, 2024.
- [27] D. Fabjan, V. Popović, D. Radišić, *et al.*, “Comparison of google maps, here maps, and openstreetmap reliability,” in *Transportation Research Procedia*, vol. 22, pp. 55–62, Elsevier, 2017. Compares data sources and reliability of major map providers.
- [28] P. Blythe, S. Edwards, and G. Hill, “Vehicle-as-a-sensor: usage of vehicle data for road weather monitoring,” *IET Intelligent Transport Systems*, vol. 10, no. 5, pp. 315–322, 2016. Validates the use of wiper and ABS data for environmental sensing.
- [29] W. Zhang and Y. Liu, “Advanced traffic flow prediction using deep learning,” *IEEE Access*, 2024.
- [30] D. Luxen and C. Vetter, “Real-time routing with openstreetmap data,” in *Proceedings of the 19th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems*, pp. 513–516, 2011. Original paper describing OSRM and its memory-intensive Contraction Hierarchies algorithm.
- [31] J. Martinez *et al.*, “Poi classification and data integration from multiple sources,” *Applied Sciences*, vol. 10, no. 17, p. 5882, 2020.
- [32] A. Pratelli *et al.*, “Sustainability of urban transport: A case study of citation analysis,” *Sustainability*, vol. 14, no. 9, p. 4985, 2022.

- [33] M. Yıldırım *et al.*, “İstanbul Çevre yollarında yağmurun serbest akım hızı ve kapasiteye etkisinin İrdelenmesi,” *ResearchGate*, 2024.
- [34] A. Mobasheri *et al.*, “Openstreetmap data quality: A systematic literature review,” *Data*, vol. 7, no. 8, p. 106, 2022.
- [35] A. Crooks *et al.*, “Big-data reliability in informing the 15-min city modelling: A comparison between openstreetmap and google maps,” in *Proceedings of the GIS Research UK (GISRUK)*, 2024.
- [36] R. Hecht, S. Hahmann, and T. Rieche, “The divergent geographies of urban amenities: A data comparison between openstreetmap and google maps,” *Preprints*, 2024. ResearchGate Preprint.
- [37] I. Akdoğan and S. Şener, “Sürdürülebilir kent için veriye erişim: İstanbul Örneği,” tech. rep., Türkiye Ekonomik ve Sosyal Etüdler Vakfı (TESEV), İstanbul, 2016.
- [38] F. Biljecki and Y. S. Chow, “Global openstreetmap vs. google maps: A comparative data quality assessment for urban amenities,” *Remote Sensing*, vol. 14, no. 19, p. 4723, 2022.
- [39] L. Leinenweber, D. Evans, and G. Hobona, “Ogc disaster pilot 2021: Engineering report,” OGC Engineering Report 21-025, Open Geospatial Consortium (OGC), 2021.
- [40] A. Mesbah, A. Van Deursen, and D. Roest, “Automated analysis of ajax user interfaces,” in *Software Engineering for Modern Web Applications: Methodologies and Technologies*, pp. 202–232, IGI Global, 2012. Explains why traditional crawlers fail on dynamic DOM/AJAX sites.

- [41] D. Glez-Peña, A. Lourenço, H. López-Fernández, M. Reboiro-Jato, and F. Fdez-Riverola, “Web scraping technologies in an api world,” *Briefings in Bioinformatics*, vol. 15, no. 5, pp. 788–797, 2014. Compares traditional scraping vs. headless browser techniques for dynamic data.
- [42] D. Shukla, C. Shivnani, and D. Shah, “Comparing oracle spatial and postgres postgis,” *International Journal of Computer Science and Communication (IJCSC)*, vol. 7, no. 2, pp. 95–100, 2016.
- [43] A. Makris, K. Tserpes, G. Spiliopoulos, and D. Anagnostopoulos, “Performance evaluation of mongodb and postgresql for spatio-temporal data,” in *Proceedings of the Workshops of the EDBT/ICDT 2019 Joint Conference (BMDA)*, vol. 2322 of *CEUR Workshop Proceedings*, CEUR-WS.org, 2019.
- [44] G. Toktomusheva, “Indexing in postgresql: Performance evaluation and use cases,” *Preprints.org*, 2025. Preprint.
- [45] TigerData, “Why postgres wins for ai and vector workloads.” <https://www.tigerdata.com/blog/why-postgres-wins-for-ai-and-vector-workloads>, jan 2025.
- [46] R. Geisberger, P. Sanders, D. Schultes, and D. Delling, “Contraction hierarchies: Faster and simpler hierarchical routing in road networks,” in *International Workshop on Experimental and Efficient Algorithms*, pp. 319–333, Springer, 2008. Foundation of the static pre-processing technique used in fast engines like OSRM.
- [47] D. Mikiewicz, A. Klimach, and I. Kaczmarek, “Comparative analysis of open source routing machine and pgrouting in the context of emergency vehicles routing,” *GIS*

- Ostrava 2017*, pp. 22–24, 2017. Highlights the trade-off between OSRM speed and pgRouting flexibility.
- [48] M. Pregnotato, A. Ford, S. M. Wilkinson, and R. J. Dawson, “The impact of flooding on road transport: A depth-disruption function,” *Transportation Research Part D: Transport and Environment*, vol. 55, pp. 67–81, 2017. Provides the mathematical model for calculating delay costs based on flood depth.
- [49] R. Sankepally and K. Rajan, “Improving path query performance in pgrouting using a map generalization approach,” *ISPRS Annals of the Photogrammetry, Remote Sensing and Spatial Information Sciences*, vol. 4, no. 4/W6, pp. 87–93, 2018.
- [50] IJSREM, “Performance and usability comparison of python web frameworks for data science application: Django, flask and fastapi,” *International Journal of Scientific Research in Engineering and Management (IJSREM)*, vol. 8, no. 1, 2024.
- [51] M. O’Dwyer and J. Smith, “Efficient data validation in python: A comparative analysis of pydantic and marshmallow,” *Journal of Software Engineering and Applications*, vol. 16, no. 4, pp. 210–225, 2023. Discusses the performance benefits of Rust-based validation cores in interpreted languages.
- [52] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, “React: Synergizing reasoning and acting in language models,” in *International Conference on Learning Representations (ICLR)*, OpenReview, 2023. Foundational paper for the cyclic reasoning loops used in agent orchestration.
- [53] A. Kaplunovich, “Langgraph-orchestrated llm agents for scalable movie knowledge graphs and question answering,” in *International Conference on AI Research (ICAIR)*, vol. 5, Academic Conferences International Ltd, 2025.

- [54] J.-R. W. Group, “Json-rpc 2.0 specification.” <https://www.jsonrpc.org/specification>, 2010. Lightweight remote procedure call protocol standard used for MCP communication.
- [55] C. Holmes, “Cloud optimized geotiff: An imagery format for cloud-native geospatial processing,” *Planet Labs Engineering Blog*, 2016. Foundational concept describing how range requests enable efficient remote pixel access.
- [56] E. Gülcüoğlu, A. B. Üstün, and N. Seyhan, “Comparison of flutter and react native platforms,” *Journal of Internet Applications and Management*, vol. 12, no. 2, pp. 129–143, 2021.
- [57] Scribd, “Comparative analysis of flutter vs react native.” <https://www.scribd.com/document/791111736/Comparative-Analysis-of-Flutter-vs-React-Native>, 2024.
- [58] B. Sharma, P. Bansal, M. Chugh, A. Chauhan, P. Anand, Q. Hua, and A. Jain, “Benchmarking geospatial database on kubernetes cluster,” *EURASIP Journal on Advances in Signal Processing*, vol. 2021, no. 1, pp. 1–29, 2021.
- [59] OpenAI, “simple-evals: Simple evaluation scripts for ai benchmarks.” <https://github.com/openai/simple-evals>, 2024.
- [60] M. Haklay, “How good is volunteered geographic information? a comparative study of openstreetmap and ordnance survey datasets,” *Environment and planning B: planning and design*, vol. 37, no. 4, pp. 682–703, 2010. Validates the use of OSM for physical infrastructure in urban areas.

- [61] P. A. Vretanos, “Web feature service implementation specification,” OGC Implementation Specification OGC 04-094, Open Geospatial Consortium, 2005.
- [62] R. Ilce and G. Ozkan, “Coordinate transformation between itrf96 and wgs84 for gis applications in turkey,” *Journal of Geodesy and Geoinformation*, vol. 6, no. 2, pp. 112–118, 2019.
- [63] J. W. Rouse, R. Haas, J. Schell, and D. Deering, “Monitoring vegetation systems in the great plains with erts,” in *Third Earth Resources Technology Satellite-1 Symposium*, vol. 1, pp. 309–317, NASA, 1974.